

Report Hubert Curien Internship

ECHE Tiago

May - July 2026

(May 26 - July 24)

Contents

1	Introduction	2
2	Notation and setup	2
3	First implementation : Conformal Learning	3
3.1	Mathematical equations	3
3.2	Pseudo-algorithm	4
3.3	Poisson distribution and adaptive learning rate	5
3.4	Smooth error function	6
4	Core-set	7
5	Another sampling rule	9
6	Heavy-tailed data	11
6.1	Conformal learning	11
6.2	Uncertainty sampling	12
7	Comparative results	13
8	Theoretical overview	14
8.1	Generality	14
8.2	Conformal learning	15
8.3	Core-set model	15
8.4	Uncertainty sampling	15
9	Conclusion	15

1 Introduction

The objective of this internship was to study ways to solve the LASSO regression in an online setting.

$$\beta^* = \arg \min_{\beta} \frac{1}{2} \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

After looking at a first implementation made by [4], we decided to create our own approach inspired by the work in [2]. The key idea was to use conformal learning to decide whether or not to train on the current data using a threshold and a score. The training is a basic gradient descent, adapted with a proximal operator to deal with the $L1$ norm on the LASSO regression. We will consider the hyperparameter λ as known in our work.

This report now aims at describing the path followed during the whole internship, with the different ideas and results encountered as well as the difficulties not solved.

To do so, we started with a first genuine implementation that we refined and modified multiple times. We then explored two sampling options to have other pathways available. Afterward we tested them in different setup to compare them before addressing the theoretical difficulties and ideas explored but left open.

2 Notation and setup

The whole data set is noted (X, Y) , with X being a matrix (T, p) , T being the number of entries and p the dimensions of the data (the number of parameters). Y is a column vector of the outputs.

We denote by (x_t, y_t) the t -th data point. We generally index by t the variable depending of the current data point. The regression parameter is β , β_t denotes the value of the parameter after training on the data t , $\beta_t^{(k)}$ represents β after the k -th iteration on the data t .

During our work we tested two similar setup : either strictly sequentially, seeing one data at a time, or by seeing a certain window of data at a time $[t - window, t]$. Generally speaking the scores are calculated on the whole control window including all the points, while we train only on the last data point on the window. We will generally omit the window part, as it didn't add anything relevant during our tests.

Our tests are done with the following setup if not stated otherwise :

$$T = 10000 \quad (1)$$

$$p = 100 \text{ the dimension of the entries} \quad (2)$$

$$X \sim \mathcal{N}(0, 1) \quad (3)$$

$$S = 20\%, \text{ the sparsity of the oracle parameter} \quad (4)$$

$$\epsilon \sim \mathcal{N}(0, 0.1) \quad (5)$$

$$\beta^* \sim \mathcal{N}(0, 1) \text{ with a sparsity level of } S \quad (6)$$

$$Y = X\beta^* + \epsilon \quad (7)$$

$$\tau, \gamma \text{ are learning rates} \quad (8)$$

$$\alpha \text{ a confidence level} \quad (9)$$

3 First implementation : Conformal Learning

3.1 Mathematical equations

As stated above, our first implementation relied on the idea of conformal learning in order to decide when to adapt our model, based on a threshold, q , and a score, s . The update of our parameter β relying on proximal gradient. Given two learning rates γ, τ , a confidence level α we have :

$$q_{t+1} = q_t + \gamma(e_t - \alpha) \quad (10)$$

$$s_t = \frac{1}{2} \|y_t - x_t \beta\|_2^2 + \lambda \|\beta\|_1 \quad (11)$$

$$\beta_t^{(k+1)} = \text{Soft}_{\lambda\tau}(\beta_t^{(k)} - \tau x_t^T (x_t \beta_t^{(k)} - y_t)) \quad (12)$$

With :

$$e_t = \begin{cases} 1 & \text{if } s_t > q_t \\ 0 & \text{if } s_t \leq q_t \end{cases} \quad (13)$$

$$\text{Soft}_{\lambda\tau}(u) = \begin{cases} u + \lambda\tau & \text{if } u < -\lambda\tau \\ u - \lambda\tau & \text{if } u > \lambda\tau \\ 0 & \text{else} \end{cases} \quad (14)$$

and $\beta_t^{(k+1)}$ being the (k+1)th iteration of β done at the data number t .

3.2 Pseudo-algorithm

Algorithm 1 Conformal PID for online LASSO

```

1:  $\lambda \leftarrow \text{fixed}$ 
2:  $\tau \leftarrow \text{fixed}$ 
3:  $\gamma \leftarrow \text{fixed}$ 
4:  $\alpha \leftarrow \text{fixed}$ 
5: function SCORE( $x, y, \beta, \lambda$ )
6:   return  $\frac{1}{2}(y - x * \beta)^2 + \lambda |\beta|_1$ 
7: end function
8:
9: function SOFT( $u, \lambda, \tau$ )
10:  if  $u \leq -\tau\lambda$  then
11:    return  $u + \tau\lambda$ 
12:  else if  $u \geq \tau\lambda$  then
13:    return  $u - \tau\lambda$ 
14:  else
15:    return 0
16:  end if
17: end function
18:
19:  $\beta \leftarrow 0$ 
20:  $q \leftarrow q_0$ 
21: for  $(x_i, y_i) \in (X, Y)$  do
22:    $\beta_{temp} \leftarrow \beta$ 
23:   if SCORE( $x, y, \beta_{temp}$ )  $\leq q$  then
24:      $q \leftarrow q - \gamma\alpha$ 
25:   else
26:      $q \leftarrow q + \gamma(1 - \alpha)$ 
27:   end if
28:   while SCORE( $x, y, \beta_{temp}$ )  $> q$  do
29:      $u = \beta_{temp} - \tau x_i^T (x_i \beta_{temp} - y_i)$ 
30:      $\beta_{temp} \leftarrow \text{SOFT}(u, \lambda, \tau)$ 
31:   end while
32:    $\beta \leftarrow \beta_{temp}$ 
33: end for

```

Although this version was working (i.e the experimental tests were showing a converging β), the iterations over time were quite unstable, and even after converging close to the optimal solution, we still had data point where we were doing a lot of iterations. We wanted to add more control to it, so we decided to use a Poisson distribution based on the score and the threshold to determine the number of iterations to do.

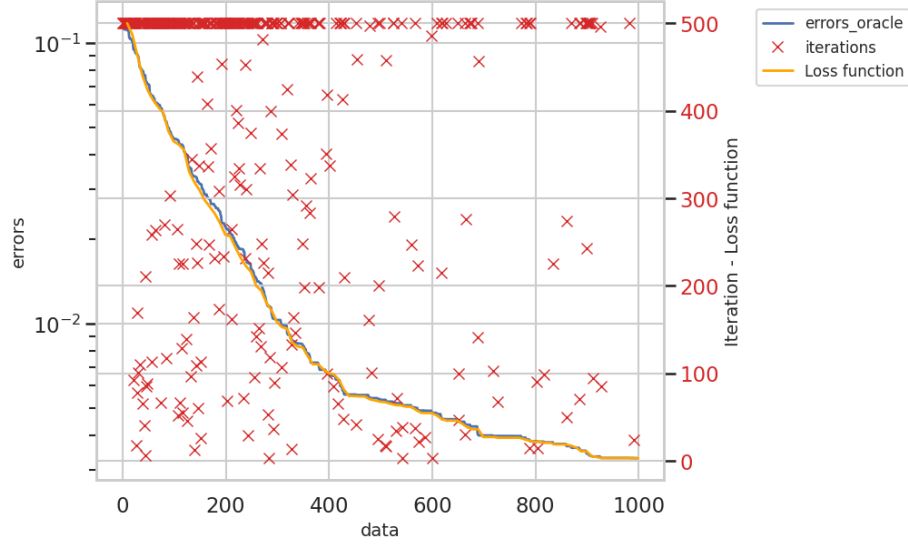


Figure 1: Plot of the Loss (orange), error with β^* (blue) and number of iteration for the t -th data point (red).

Left axis is for the blue plot and right axis for the red plot
No axis for the LOSS, we only look at the behavior

3.3 Poisson distribution and adaptive learning rate

We decided to use a distribution so that the number of iteration were strictly 0 when $s_t < q_t$ and it follows a Poisson distribution depending of s_t, q_t if $s_t \geq q_t$. i.e $N_t \sim \mathcal{P}(\phi(s_t, q_t))$. After different tries, we stuck to $\phi(s_t, q_t) = \sqrt{(s_t - q_t)/q_t}$

Another change made to add more control to the gradient descent was to use an adaptive learning rate, based on the eigenvalues of $x_t^T x_t$. We chose $eigenvalue_t = \max(\max(eigenvalues(x_t^T x_t)), eigenvalue_{t-then1})$ and $\tau_t = 0.9/eigenvalue_t$

Algorithm 2 Poisson distribution

```

1: ...
2:  $N \leftarrow \mathcal{P}(\sqrt{\max(\frac{s-q}{q}, 0)})$ 
3: for n=1 to N do
4:   ...
5: end for
```

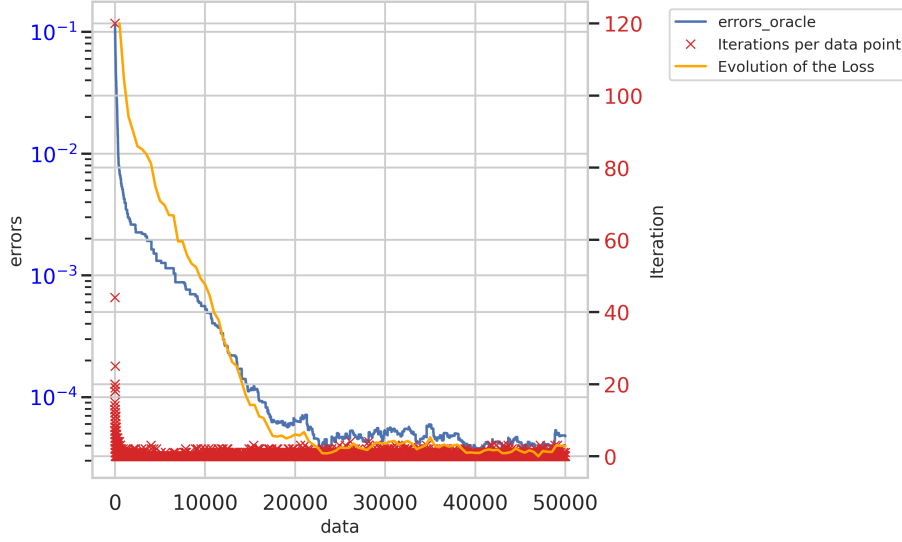


Figure 2: Plot of the Loss (orange), error with β^* (blue) and number of iteration per data point (red).

Here we use a Poisson distribution to set the number of iteration done for t-th data point

No axis for the LOSS, we only look at the behavior

Those changes gave us a much more smooth evolution of the number of iteration on the i-th data point, and we also achieved a better precision at the end.

That was already a better result, but we decided to explore different paths to look for other options alongside that searching for theoretical results about our model. The guiding thought was to adapt our decision about training or not on the current data point.

We will now present those different changes and what inspired them. Unfortunately for now we don't have theoretical guarantee on the convergence of the algorithm.

3.4 Smooth error function

Until now the update of the threshold q depended on an error function e_t which was binary : either 1 or 0. We decided to smoothen it to take into account the distance to the threshold in its update. We also thought that it could help us to see the update of q_t as a sort of gradient descent and so smoothening the "gradient" could make it easier to study.

To do so we used the following update :

$$q_{t+1} = q_t + \gamma \tanh(q - s)|q - s|$$

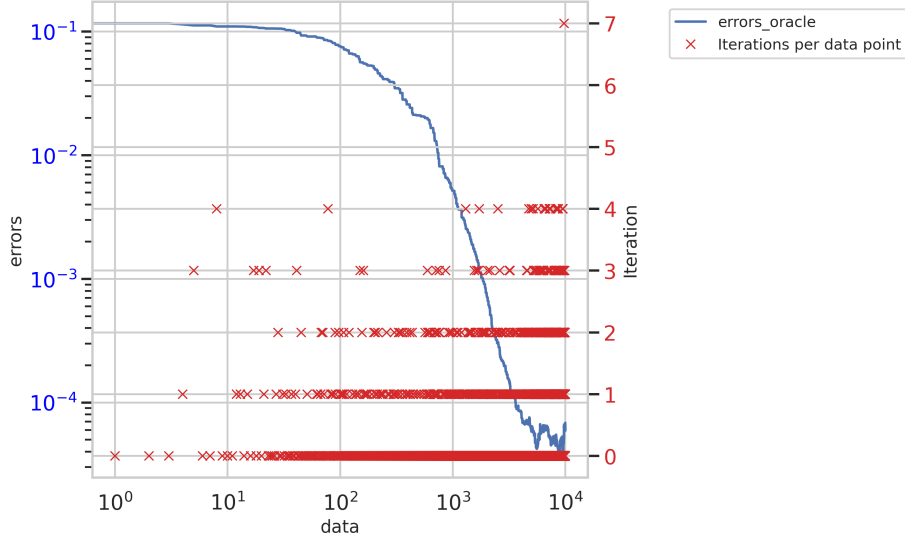


Figure 3: Plot of the error and the iteration per data on a log scale with a smoother conformal learning

We do much fewer iteration per data, even though it's scattered along the whole set of data. We still achieve a good precision, around 10^{-5} at the end of the training for less than 0.15 iterations per data in average at the end.

We weren't quite sure on how to approach this model theoretically. Our best intuition was to model it as an dynamic system but that seemed quite complicated. Before going deeper into a theoretical analysis we decided to explore new ideas, with the hope of finding a model that would achieve similar results but with a more obvious theoretical guarantee.

4 Core-set

An idea we tried to explore was to replace the conformal learning part by something inspired by the core-set theory [1] : We try to estimate how important is the current data point regarding what we have already seen, and set our number of iteration based on the importance or rarity. The idea comes from the core-set theory where you use a weight function on your training set to sample it and obtain a smaller training set that will be equivalent as the whole one. The weight function as to be more important on data point that could have an important impact on your training algorithm, and remains low for less impacting point. That can be linked with the idea of rarity : if a point is rare, different than the other ones, missing it can lead to an important error. For a more common point, even if you miss it, you will still find similar point latter (or have already seen similar point), on which you can still train and have a similar result.

To implement that we kept a running weighted mean of the scores, and for each

new data point we weighted it by the distance between it score and the running mean.

$$Sum_t = \sum_{i=1}^t w_t s_t \quad (15)$$

$$W_t = \sum_{i=1}^t w_t \quad (16)$$

$$w_t = \left| \frac{Sum_t}{W_t} - w_t s_t \right|, \text{ we solve this implicit equation :} \quad (17)$$

$$\Rightarrow \left(\frac{Sum_t}{W_t} - w_t s_t \right)^2 - w_t^2 = \left(\frac{Sum_t}{W_t} - (s_t + 1)w_t \right) \left(\frac{Sum_t}{W_t} - (s_t - 1)w_t \right) = 0 \quad (18)$$

$$\Rightarrow \begin{cases} w_t = \frac{Sum_t}{W_t} \frac{1}{s_t + 1} \\ w_t = \frac{Sum_t}{W_t} \frac{1}{s_t - 1} \end{cases} \quad (19)$$

to ensure w_t being positive and avoid the denominator being 0 we chose the first solution for w_t . Indeed s_t could be less or equal to one, making the second solution impossible.

Algorithm 3 Core-set setup

```

1: ...
2:  $Sum \leftarrow 0$ 
3:  $W \leftarrow 1$ 
4: for  $(x, y) \in (X, Y)$  do
5:    $s \leftarrow \text{SCORE}(x, y, \beta)$ 
6:    $w_t \leftarrow \frac{Sum}{W} \frac{1}{s+1}$ 
7:    $W \leftarrow W + w_t$ 
8:    $Sum \leftarrow Sum + w_t \cdot s$ 
9:    $N \leftarrow \mathcal{P}(\sqrt{|w_t \cdot s - \frac{Sum}{W}|})$ 
10:  for  $n=1$  to  $N$  do
11:    ...
12:  end for
13: end for

```

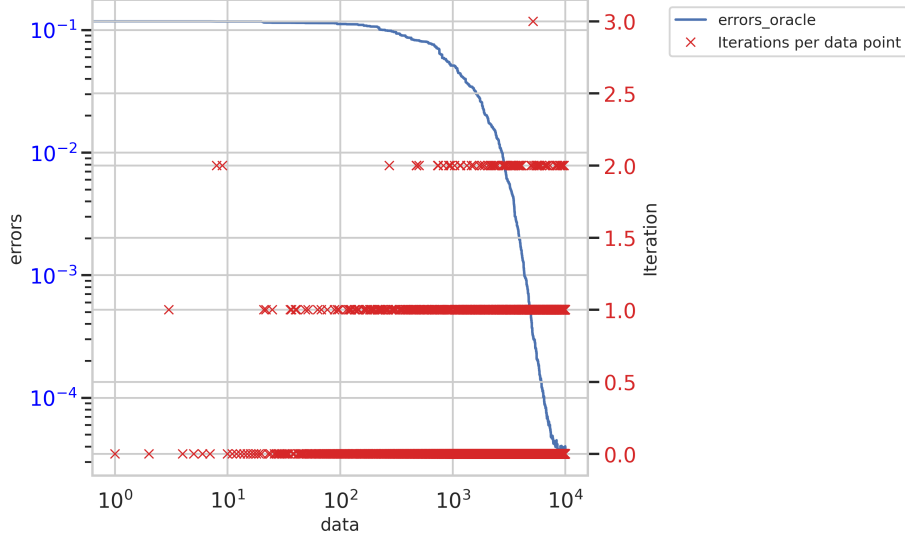


Figure 4: Plot of the error with β^* and iteration per data point on a log scale

We observe a similar behavior than with the conformal learning setup, with a maximum amount of iterations on one point a bit lower than with the previous setup (3 vs 7). We achieve a similar precision, with a bit more of total iterations but still below 0.2 per data.

5 Another sampling rule

We also took a look at the work done at [5] to decide whether or not to train on the data point. This time the answer is binary : either we train one time or we don't train at all. To do so we fixed a budget at 25% of the total of the amount of data. We used $u_t = |y_t - x_t\beta|$ as the uncertainty on our model. with N being the total of data we will train on and n_{seen} the number of data we have trained on. The probability is then computed as follow :

$$\pi_{base_t} = \min \left(\frac{u_t * budget * (t+1)}{N \sum_{i=1}^t u_i}, \frac{budget * t}{N} - n_{seen} \right)_{[0,1]} \quad (20)$$

$$\pi_t = 0.5 * \pi_{base_t} + 0.5 * \frac{budget}{N} \quad (21)$$

Here we clip π_{base_t} to $[0, 1]$ in order to have a probability

Algorithm 4 Uncertainty sampling

```

1:  $E_u \leftarrow 0$ 
2:  $n_{seen} \leftarrow 0$ 
3:  $N \leftarrow |Y|_0$ 
4:  $\dots$ 
5: for  $t = 0 : (N - 1)$  do
6:    $x \leftarrow X[t]$ 
7:    $y \leftarrow Y[t]$ 
8:    $u \leftarrow |y - x\beta|$ 
9:    $E_u \leftarrow E_u + u$ 
10:   $\pi \leftarrow \max(0, \min(\min(\frac{u * budget * (t+1)}{N E_u}, \frac{budget * t}{N} - n_{seen}), 1))$ 
11:   $\pi \leftarrow 0.5\pi + 0.5\frac{budget}{N}$ 
12:   $I \sim \mathcal{B}(\pi)$ 
13:  for  $n=1$  to  $I$  do
14:     $\dots$ 
15:  end for
16:   $n_{seen} \leftarrow n_{seen} + 1$ 
17: end for

```

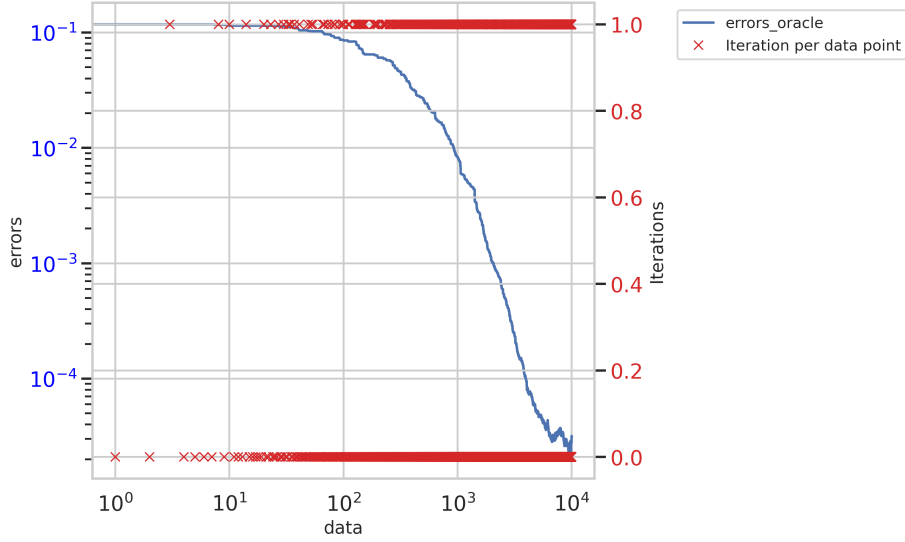


Figure 5: Plot of the error with β^* and iteration per data point on a log scale

We reach a similar behavior than the previous setups, we continue to reach a good precision around 10^{-5} for less than 0.2 iterations per data point. Therefore we decided to test different setup to see how would those models would react to heavy-tailed data-set.

6 Heavy-tailed data

To study this setup we used student's distributions with a varying degree of freedom, as this can make the distribution vary between a normal distribution (when we have a lot of degrees of freedom) and a Cauchy distribution (very heavy tail) when we have 1 degree of freedom.

Generally speaking, those three models perform poorly for degree of freedom close to 1. We only achieve an error with β^* around 0.1 at the end. Nevertheless the number of iterations doesn't increase. We then tried to make the models adapt more to the distribution, and increase the iterations to have a better model.

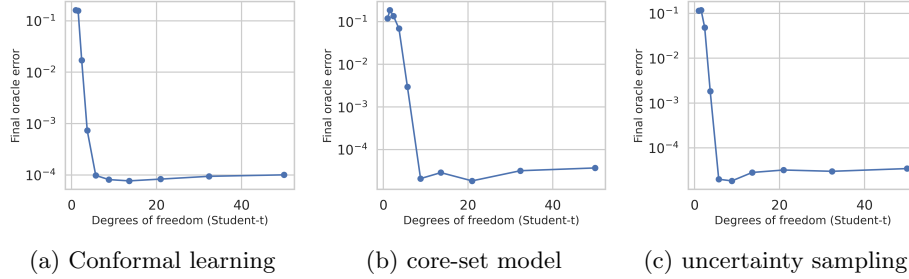


Figure 6: Last error with β^* depending on the degree of freedom of the student law.

To reduce computing time we took only 3000 data point for each training

6.1 Conformal learning

To try getting a better conformal learning model we decided to re-introduce a confidence level parameter α instead of just using a tanh function.

$$q_{t+1} = q_t - \left(\frac{1}{2} + \frac{1}{2} \tanh(s_t - q_t) - \alpha \right) |s - q|$$

The idea was to see how the confidence level would modify the behavior of our model and how this would interact with a more or less heavy tailed input. For a confidence level of 0.05, although this gave us good results for a student distribution with a degree of freedom below 2, this also gave us worse result for less heavy tailed distributions (a degree of freedom above 10). That could mean that we should try to adapt this parameter to the (expected) distribution, but it could be hard to know or predict it.

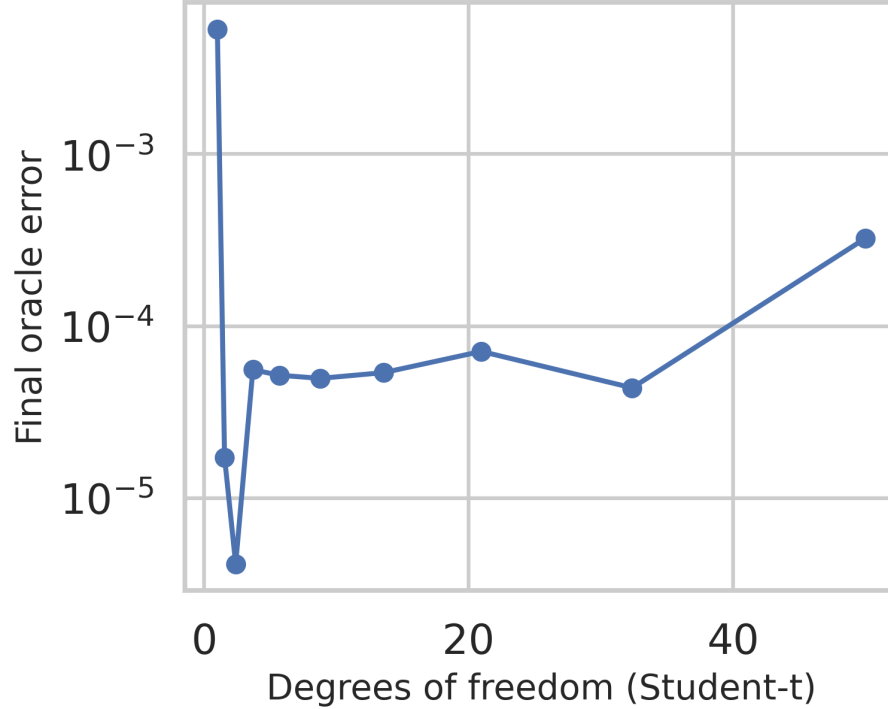


Figure 7: Error with β^* depending on the degree of freedom of the student distribution. Conformal learning based model, with a confidence level $\alpha = 0.05$

We can see that it performs better than the previous version for heavy tailed distribution. However, if it seems to perform at a similar level for medium tailed distribution, it is a bit worse for a distribution close to a normal one.

6.2 Uncertainty sampling

For this model, increasing our budget is an easy way to increase the number of iterations, but we also have to be careful as more iteration isn't a guarantee of a better model. We have to question whether it is better to iterate more on specific data point, or if it is better to scatter our iteration along the whole set. Typically, for a student distribution with two degree of freedom we reach a precision of 0.079 for a budget of one fourth of the number of data point (10000 data points in total) and we reach 0.055 for a "full" budget, i.e 10000 iterations of budget. Although we multiplied by four our budget, the precision at the end didn't change much.

7 Comparative results

During our tests we noticed another interesting setup which is a variation of the conformal learning without alpha with this update of our threshold :

$$q_{t+1} = q_t + \gamma \tanh(q - s)$$

So we removed the $|s - q|$ part This tend to be a bit better for heavy tailed distributions in term of precision but worse than the version with alpha. Moreover, it also requires more iterations than the classical conformal learning one.

We will now present a table studying the four models for different student distributions of the inputs, in order to see how they perform for heavy to lightly tailed distributions. We will look at three characteristics : The finale error to β^* (the minimizer of the global LOSS), the total number of iterations made, and the maximal error made on a data point : $\max_{x,y \in (X,Y)} |y - x\beta|$. The tests have been made with 10000 data points as usual and a fixed seed, the same for all models and all student distributions. We only made a single run for each model and each distribution.

Table 1: Comparison of methods across degrees of freedom

Degree of freedom : 1.00			
Model	Final error β^*	Total iterations	$\max_{x,y \in (X,Y)} y - x\beta $
Conformal (w/o alpha, with $ s - q $)	$1.44 \cdot 10^{-1}$	$1.94 \cdot 10^3$	$5.59 \cdot 10^4$
Conformal (w/o alpha, w/o $ s - q $)	$1.34 \cdot 10^{-2}$	$1.40 \cdot 10^5$	$5.06 \cdot 10^3$
Conformal (with alpha)	$4.14 \cdot 10^{-5}$	$1.51 \cdot 10^7$	420
Core-set sampling	$1.80 \cdot 10^{-1}$	$1.80 \cdot 10^3$	$7.85 \cdot 10^4$
Uncertainty sampling	$1.69 \cdot 10^{-1}$	$1.72 \cdot 10^3$	$7.34 \cdot 10^4$
Degree of freedom : 1.54			
Conformal (w/o alpha, with $ s - q $)	$1.44 \cdot 10^{-1}$	$1.94 \cdot 10^3$	982
Conformal (w/o alpha, w/o $ s - q $)	$4.79 \cdot 10^{-3}$	$1.39 \cdot 10^4$	548
Conformal (with alpha)	$9.71 \cdot 10^{-8}$	$2.18 \cdot 10^6$	1.21
Core-set sampling	$1.46 \cdot 10^{-1}$	$1.91 \cdot 10^3$	$2.61 \cdot 10^3$
Uncertainty sampling	$7.21 \cdot 10^{-2}$	$2.25 \cdot 10^3$	832
Degree of freedom : 3.68			
Conformal (w/o alpha, with $ s - q $)	$4.02 \cdot 10^{-5}$	$1.90 \cdot 10^3$	$6.48 \cdot 10^{-1}$
Conformal (w/o alpha, w/o $ s - q $)	$1.4 \cdot 10^{-5}$	$2.89 \cdot 10^3$	$5.38 \cdot 10^{-1}$
Conformal (with alpha)	$3.31 \cdot 10^{-5}$	$5.41 \cdot 10^5$	1.09
Core-set sampling	$1.03 \cdot 10^{-1}$	$2.05 \cdot 10^3$	30.1
Uncertainty sampling	$3.41 \cdot 10^{-3}$	$1.94 \cdot 10^3$	5.68
Degree of freedom : 8.79			

Continued on next page

Table 1 – continued from previous page

Model	Final error β^*	Total iterations	$\max_{x,y \in (X,Y)} y - x\beta $
Conformal (w/o alpha, with $ s - q $)	$2.90 \cdot 10^{-5}$	$1.51 \cdot 10^3$	$4.46 \cdot 10^{-1}$
Conformal (w/o alpha, w/o $ s - q $)	$3.05 \cdot 10^{-5}$	$2.54 \cdot 10^3$	$4.32 \cdot 10^{-1}$
Conformal (with alpha)	$3.41 \cdot 10^{-5}$	$3.17 \cdot 10^5$	$4.73 \cdot 10^{-1}$
Core-set sampling	$2.23 \cdot 10^{-5}$	$1.62 \cdot 10^3$	$4.54 \cdot 10^{-1}$
Uncertainty sampling	$1.44 \cdot 10^{-5}$	$1.62 \cdot 10^3$	$4.84 \cdot 10^{-1}$
Degree of freedom : 21.0			
Conformal (w/o alpha, with $ s - q $)	$4.23 \cdot 10^{-5}$	$1.49 \cdot 10^3$	$4.48 \cdot 10^{-1}$
Conformal (w/o alpha, w/o $ s - q $)	$5.50 \cdot 10^{-5}$	$2.51 \cdot 10^3$	$4.93 \cdot 10^{-1}$
Conformal (with alpha)	$5.95 \cdot 10^{-5}$	$3.14 \cdot 10^5$	$6.34 \cdot 10^{-1}$
Core-set sampling	$2.20 \cdot 10^{-5}$	$2.01 \cdot 10^3$	$4.59 \cdot 10^{-1}$
Uncertainty sampling	$2.27 \cdot 10^{-5}$	$1.61 \cdot 10^3$	$4.44 \cdot 10^{-1}$
Degree of freedom : 50.0			
Conformal (w/o alpha, with $ s - q $)	$3.67 \cdot 10^{-5}$	$1.53 \cdot 10^3$	$5.22 \cdot 10^{-1}$
Conformal (w/o alpha, w/o $ s - q $)	$4.43 \cdot 10^{-5}$	$2.59 \cdot 10^3$	$4.66 \cdot 10^{-1}$
Conformal (with alpha)	$1.98 \cdot 10^{-4}$	$6.60 \cdot 10^5$	$7.14 \cdot 10^{-1}$
Core-set sampling	$3.38 \cdot 10^{-5}$	$2.22 \cdot 10^3$	$4.54 \cdot 10^{-1}$
Uncertainty sampling	$3.18 \cdot 10^{-5}$	$1.65 \cdot 10^3$	$4.64 \cdot 10^{-1}$

Note: All the code for the different models can be found here : <https://gitlab.com/echetiago/internship-online-lasso>

We can see that the alpha version of conformal learning is the best one for heavy tailed inputs but the trade-off is a lot of iterations. Even for lightly tailed distributions we continue to have a lot of iterations.

The conformal learning version without $|s - q|$ is a sort of in between, it worse than the previous one, but better than the other three. We again see quite a lot of iterations, but it decrease to reach a similar order of magnitude than the first version of conformal learning for distributions closer to a normal one.

After that, the core-set, the uncertainty sampling and the first conformal learning performs at similar level for all student distributions, with the core-set being the worse one with more iterations and a precision that increase slower.

8 Theoretical overview

8.1 Generality

Two main points are constant for all models : How to spread our training/iterations on our data points and can we make link with offline setup, where the convergence of the model is proven or directly prove the convergence?

The first point leads to this question : how does training on one data point help us finding the minimizer of our global function.

Indeed, on each point we use a gradient descent using a "local" version of the loss of the LASSO regression :

$$l_t(\beta) = \frac{1}{2} |y_t - x_t \beta|_2^2 + \lambda |\beta|_1$$

We know that if we train enough we can be as close as we want to a minimizer (not necessary unique) of this local loss, but it's unclear how this solution is linked to the global solution or the other local solutions. Something that could be interesting would be to prove that we can at least control the worst local function. That mean, finding β_T so that we can control $\max_{t \in [1, T]} l_t(\beta_T)$. By doing so and using λ/T in the local losses we can get an upper bound of the global LOSS because we can write it as $L(\beta) = \sum_{t=1}^T l_t(\beta) \leq T \cdot \max_{t \in [1, T]} l_t(\beta)$. To get such a bound we would need to know how training on one data point will affect the training already done on the previous ones. i.e if we have $t_1, t_2, \epsilon_1, \epsilon_2, \tilde{\beta}_{t_1}, \tilde{\beta}_{t_2}$ so that

$$\begin{cases} |\beta_{t_1}^* - \tilde{\beta}_{t_1}| \leq \epsilon_1 \\ |\beta_{t_2}^* - \tilde{\beta}_{t_2}| \leq \epsilon_2 \end{cases} \quad (22)$$

Then can we say something about $|\beta_{t_1}^* - \tilde{\beta}_{t_2}|$ and vice versa ? Can we ensure that the training on the data point t_2 won't affect too much what we already had on the t_1 -th and can we estimate it ?

This consideration indicate that it's not a good idea to spend too much iteration on one data point, as being close to the local minimizer will either be counterproductive for other data point, or be forgotten on the following training. The second question is of course model dependent as they aren't motivated by the same ideas.

8.2 Conformal learning

This model isn't inspired by an existing offline model. Here our main ideas was to treat it as a dynamic system, something already studied in machine learning [3], and maybe using Lyapunov approach. To do so the tanh function used on the threshold update can be helpful as it permit to treat this update as a sort of gradient descent. However the ways to fully model the algorithm and then to find a Lyapunov function proving the convergence remain unclear and we couldn't go far on this path.

8.3 Core-set model

This one is an adaptation of the core-set theory and aims at getting a similar sampling/weights as the one we could get with an offline setup. Nevertheless, here our weighting function is based on the previous observation and it's unclear how this could be linked to a sampling done in an offline setup. Additionally, to make a clear link we would need to use the same training on the offline sampling and in the online sampling, but to take advantage of our online setup we need to train during the sampling. So we either have to show how the offline and online training could be related or directly show that the online training can guarantee the convergence. Because of those limitations, it seems hard to make good progress with this model, and because it seems to be the worst model in practice, it may not be worth it.

8.4 Uncertainty sampling

This model is not driven by an offline algorithm, and it aims at using another rule to decide when we shall train. Given that, it resembles the conformal learning version. By using uncertainty we can hope that we will guarantee to reduce the local loss on uncertain data points. But again we reach the same difficulty discussed in generality, as we have to prove that reducing the loss on those points will, at least, not disrupt the loss on the previous ones.

9 Conclusion

We studied three different methods to solve the LASSO regression in an online setup, a conformal learning inspired one, a core-set derived one and a sort of uncertainty sampling one. Additionally, the conformal learning setup has three different variations. The general conclusion would be that all of those perform quite well for lightly tailed entries, with a good balance between being close to the solution and doing few iterations. The first conformal

learning and the uncertainty sampling being the best at it. However, in case of heavy tailed setup, the two other variations of conformal learning are the only ones that achieve somehow good results. The alpha-version is by far the best but at the cost of an enormous amount of iterations, three order of magnitude above the number of data points, and the intermediate version give a sort of compromise. This version, driven by $q_{t+1} = q_t + \gamma \tanh(q - s)$ is a good intermediate, because even though it reaches more iterations in heavy tailed setup, this is expected as the distribution of the data make the model harder to optimize. However it decreases as we reach lighter tailed distribution and end up within the same order of magnitude as the first conformal learning model.

The uncertainty model behave in a similar way as the first conformal learning one, given that it at least give another possibility for a theoretical study and it might be possible to adapt it to achieve better results for heavy tailed entries.

The core-set inspired setup seems less promising, both theoretically and in practice. Unless one can find a way to give theoretical guarantee, it's for now the least interesting setup.

References

- [1] Dan Feldman. *Introduction to Core-sets: an Updated Survey*. 2020. arXiv: 2011.09384 [cs.LG]. URL: <https://arxiv.org/abs/2011.09384>.
- [2] Ben Gao et al. *Conformal Online Learning of Deep Koopman Linear Embeddings*. 2026. arXiv: 2511.12760 [cs.LG]. URL: <https://arxiv.org/abs/2511.12760>.
- [3] Bryan Van Scoy and Laurent Lessard. *A Tutorial on a Lyapunov-Based Approach to the Analysis of Iterative Optimization Algorithms*. 2023. arXiv: 2309.11377 [math.OC]. URL: <https://arxiv.org/abs/2309.11377>.
- [4] Shuoguang Yang et al. *Online Linearized LASSO*. 2023. arXiv: 2211.06039 [stat.ML]. URL: <https://arxiv.org/abs/2211.06039>.
- [5] Tijana Zrnic and Emmanuel J. Candès. *Active Statistical Inference*. 2026. arXiv: 2403.03208 [stat.ML]. URL: <https://arxiv.org/abs/2403.03208>.